Migration vers une architecture de rendu multi-moteur

Brief de cadrage | Dépôts Git, packages NPM, runtime unique et composants MCP | Révision du 10 novembre 2026

Décision proposée

Organiser l'écosystème autour de cinq dépôts Git structurants, sans créer un dépôt par package NPM. Les modèles, données et algorithmes restent indépendants des moteurs 3D. Un monorepo de rendu publie les contrats graphiques, les kernels WebGL et WebGPU, puis les adaptateurs Babylon.js et Three.js. Le modèle physique de l'atmosphère appartient à celestial-core ; son rendu est une fonctionnalité générique intégrée par chaque adaptateur de moteur.

Objectif et périmètre

La migration rend les algorithmes de tuilage, de terrain et de rendu réutilisables dans plusieurs moteurs. Un seul runtime 3D Tiles assure la sélection, le raffinement, le streaming et le cycle de vie des contenus. Les pyramides DEM et imagerie Mercator ou cube sont exposées comme des Tilesets implicites. Les adaptateurs Babylon.js, Three.js ou autres traduisent les ports communs en objets et ressources propres au moteur.

Le brief couvre les frontières de dépôts, les packages, les dépendances, les composants MCP optionnels et l'ordre de migration. Il constitue une proposition à valider avant implémentation. Les corrections terrain et runtime reprennent les décisions de cadrage ; les noms et outils MCP ci-après sont des propositions de conception.

Cible Git

Dépôt	Responsabilité	Statut / limite
celestial-core	Corps célestes, référentiels propres aux corps et modèle physique de l'atmosphère.	Dépôt existant. Pas de dépendance à un moteur graphique.
geodesy-ts	Transformations géodésiques, conversions de référentiels et projections.	Dépôt existant. Pas de rendu ni de dépendance Babylon/Three.
3d-tiles-core	Modèle 3D Tiles, runtime unique, disponibilités et streaming ; primitives de tuilage génériques dans @spacexr/tiles.	Dépôt existant. Ne contient pas la représentation spécialisée du terrain DEM.
terrain-core	Rasters, DEM, grilles CPU et sources terrain compatibles avec les Tilesets implicites du runtime commun.	Dépôt local initialisé, sans implémentation. Pas de ressources ni objets moteur.
spacexr-rendering	Contrats de rendu, algorithmes communs, kernels WebGL/WebGPU, adaptateurs moteurs et guides.	Nouveau monorepo NPM. Les packages internes peuvent être publiés séparément.

Quantia reste un dépôt transverse séparé, consommé par les projets qui en ont besoin. Il ne fait pas partie des cinq dépôts structurants de cette migration. Le total est donc de cinq dépôts principaux, ou six en comptant quantia.

Packages NPM visés

Conserver les identités NPM existantes, notamment @spacexr/geodesy, @spacexr/3d-tiles-core, @spacexr/3d-tiles-runtime, @spacexr/tiles, @spacexr/raster, @spacexr/dem, @spacexr/grid et @spacexr/3d-tiles-babylon. Les nouveaux noms restent des candidats jusqu'à la vérification des publications. Chaque composant peut être accompagné d'un package optionnel portant le suffixe -mcp.

Dépôt	Packages NPM candidats
celestial-core	@spacexr/celestial-core
geodesy-ts	@spacexr/geodesy
3d-tiles-core	@spacexr/3d-tiles-core ; @spacexr/3d-tiles-runtime ; @spacexr/tiles
terrain-core	@spacexr/raster ; @spacexr/dem ; @spacexr/grid ; autres surfaces seulement si nécessaires
spacexr-rendering	@spacexr/render-contracts · @spacexr/render-algorithms · @spacexr/render-kernels-webgl · @spacexr/render-kernels-webgpu · @spacexr/babylon-tiles3d · @spacexr/babylon-terrain · @spacexr/babylon-atmosphere · @spacexr/three-tiles3d · @spacexr/three-terrain · @spacexr/three-atmosphere

Ne publier un package que lorsqu'il possède une API stable et une frontière utile aux consommateurs. Les packages d'intégration moteur peuvent rester dans le monorepo spacexr-rendering tout en ayant des versions NPM indépendantes. Éviter les paquets vides créés uniquement pour compléter l'arborescence.

Frontières fonctionnelles

Tuilage et terrain

3d-tiles-core conserve le modèle, les codecs et le runtime unique, ainsi que @spacexr/tiles : adresses, pyramides génériques, voisinage et topologies Mercator/cube. terrain-core reçoit @spacexr/raster, @spacexr/dem et @spacexr/grid, les providers et les sources terrain compatibles avec les Tilesets implicites. ImplicitTilesetDecorator reste une passerelle du runtime commun. La représentation implicite peut être virtuelle, sans génération préalable de fichiers tileset et subtree. Les raccords entre faces du cube et les niveaux de détail utilisent la même traversée et les mêmes politiques de sélection. Aucun sélecteur terrain parallèle n'est créé.

Atmosphère

celestial-core porte la définition physique de l'atmosphère : composition, profil de densité, hauteur d'échelle et paramètres de diffusion. spacexr-rendering porte les algorithmes et les kernels de rendu. babylon-atmosphere et three-atmosphere relient ces calculs aux matériaux et cycles de rendu de leur moteur. Aucun dépôt atmosphere-core séparé n'est requis à ce stade.

WebGL et WebGPU

Les contrats et le comportement attendu des algorithmes sont communs. Les implémentations de kernels restent distinctes : GLSL pour le chemin WebGL, WGSL pour le chemin WebGPU. Les adaptateurs de moteur consomment le backend disponible et traduisent les ressources ou appels spécifiques au moteur. Ne pas promettre une source shader universelle si les APIs exigent des bindings ou des pipelines différents.

Dépendances autorisées

Projet	Peut dépendre de	Ne dépend pas de
celestial-core	Types mathématiques de base si nécessaire	Babylon.js, Three.js, WebGL, WebGPU
geodesy-ts	Types mathématiques de base	Moteurs 3D et modules de rendu
3d-tiles-core	geodesy-ts et primitives @spacexr/tiles hébergées dans ce dépôt	Babylon.js, Three.js, packages DEM/raster/grid de terrain-core
terrain-core	geodesy-ts, celestial-core selon besoin, @spacexr/tiles et contrats 3D Tiles/runtime	Moteurs graphiques, shaders et sélection concurrente du runtime
render-contracts / algorithms	Projets de domaine via interfaces stables	Babylon.js et Three.js
kernels WebGL / WebGPU	Contrats et algorithmes communs	Objets de scène Babylon.js ou Three.js
Adaptateurs moteur	Contrats, kernels, projets de domaine et moteur ciblé	Autres adaptateurs moteur

Plan de migration

1. Inventorier le code actuel. Classer chaque module, type, shader et dépendance selon sa responsabilité : 3D Tiles, terrain, algorithme graphique, WebGL/WebGPU ou moteur.
2. Établir les règles de dépendance et les contrats publics. Identifier les imports Babylon.js/Three.js présents dans les projets censés rester génériques, sans modifier encore le comportement.
3. Extraire terrain-core. Déplacer raster, DEM, grid, providers et métadonnées terrain, en conservant les identités NPM. Maintenir @spacexr/tiles et le runtime commun dans 3d-tiles-core ; préserver les passerelles existantes des pyramides vers les Tilesets implicites.
4. Créer les contrats de rendu et les algorithmes portables dans spacexr-rendering. Définir les entrées et sorties des kernels, les formats de ressources et la gestion des origines/coordonnées nécessaires au rendu planétaire.
5. Séparer les kernels par API graphique. Porter les algorithmes existants en GLSL et WGSL avec une sémantique documentée et des paramètres cohérents.
6. Migrer d'abord l'adaptateur Babylon.js existant et éprouver rapidement les mêmes contrats avec un chemin Three.js minimal. Terrain et contenus 3D passent par le même runtime. L'atmosphère suit les conventions spatiales communes et son propre cycle graphique.
7. Publier les packages, documenter les chemins de migration et maintenir des réexports temporaires depuis les anciens points d'entrée si des consommateurs en dépendent.

Critères de sortie

- Aucun import Babylon.js ou Three.js ne subsiste dans celestial-core, geodesy-ts, tiles3d-core, terrain-core ou les contrats/algorithmes génériques.
- Les pyramides DEM et imagerie Mercator et cube sont compatibles avec les Tilesets implicites et sélectionnées par le runtime 3D Tiles unique, sans obligation de générer préalablement leurs documents sur disque.
- Terrain et contenus 3D coexistent dans une scène avec la même implémentation du runtime ; plusieurs instances peuvent gérer des couches ou des corps distincts. Aucun runtime ni sélecteur terrain concurrent n'est introduit.
- Le modèle physique de l'atmosphère est défini une fois dans celestial-core et consommé par les intégrations Babylon.js et Three.js.
- Les kernels WebGL et WebGPU exposent des contrats cohérents, même lorsque leurs sources de shaders et leurs bindings diffèrent.
- Chaque package NPM documente son rôle, ses dépendances, ses moteurs requis et son chemin d'import public.

Décisions à garder ouvertes

Avant la première extraction, confirmer le scope NPM déjà utilisé et l'organisation des versions. Le choix de séparer les packages dans spacexr-rendering peut évoluer avec l'usage : garder plusieurs packages dans le monorepo, mais commencer avec un petit nombre de surfaces publiques. Les algorithmes, les contrats et les adaptateurs doivent rester séparés même si certains sont temporairement livrés ensemble.

Risques à piloter

Risque	Réponse de conception
Fuite d'API moteur dans les packages génériques	Garder les types moteurs dans les adaptateurs et faire passer les appels par des ports étroits.
Divergence entre WebGL et WebGPU	Documenter un comportement commun et maintenir les kernels GLSL et WGSL comme implémentations distinctes.
Confusion entre terrain et 3D Tiles	Conserver les données spécialisées du terrain et les exposer au runtime unique par une représentation en Tileset implicite.
Trop de packages NPM trop tôt	Garder les modules dans le monorepo, publier seulement les API stables et utiles.

Premier lot de migration

- Cartographier les imports, types et shaders existants dans tiles3d-core.
- Lister les consommateurs actuels et préserver leurs points d'entrée pendant la transition.
- Créer le squelette terrain-core et le monorepo spacexr-rendering avec leurs règles de dépendance.
- Déplacer un chemin terrain de bout en bout, puis valider la forme des contrats avant d'étendre l'extraction.

Composants MCP et behaviors

Chaque composant fonctionnel fournit une surface MCP optionnelle, distribuable sous la forme @spacexr/<package>-mcp. Cette surface s'appuie sur @cyanmycelium/mcp-core et expose les capacités réelles du composant à un agent. Le suffixe -mcp désigne un package compagnon ; le caractère @ reste réservé au scope NPM ou au séparateur de version dans une commande d'installation.

Séparation des responsabilités

Le package métier conserve ses API TypeScript et ses ports, sans dépendance MCP. Son package compagnon déclare les behaviors, schémas JSON, ressources, templates et grammaires. Un adaptateur MCP relie ces contrats à une instance existante du composant. Il appelle les mêmes API que l'application et ne réimplémente aucun calcul, chargement ou moteur de sélection.

Un behavior décrit les capacités d'une catégorie de composant. Les instances sont identifiées par des URI et des identifiants explicites. Introduire un behavior par composant fonctionnel pertinent, pas par classe, fichier ou objet de scène. Plusieurs behaviors peuvent être livrés dans un même package compagnon.

Le terme adaptateur MCP désigne le binding entre outils et API métier. Il est distinct de l'adaptateur de rendu Babylon ou Three. Les behaviors génériques ne manipulent aucun Scene, Mesh, Object3D, GPUDevice ou handle graphique sérialisé.

Dépendances et composition

Dépendances autorisées : package compagnon MCP vers package métier, ports publics et mcp-core. Le package métier n'importe jamais son compagnon. Les intégrations MCP Babylon et Three peuvent importer leur moteur ; les behaviors communs dépendent uniquement des ports de rendu indépendants du moteur.

Explorer ou une application hôte enregistre les behaviors autour de ses instances vivantes. Les packages compagnons ne démarrent ni serveur, ni transport, ni connexion par effet de bord. Le transport et la composition du serveur restent au niveau de l'hôte ; un serveur MCP est créé par session selon le modèle de mcp-core.

Le runtime 3D Tiles reste unique. Un RuntimeBehavior inspecte ou configure ce runtime ; TerrainBehavior et les behaviors de pyramide exposent leurs données et paramètres en utilisant ses ports. Aucune commande MCP ne crée un sélecteur terrain concurrent.

Packages MCP par composant

Noms candidats conservant le nom du package métier et ajoutant -mcp. Une ligne définit une responsabilité cible ; la publication intervient lorsque son API est utile et réellement implémentée. Les behaviors de composants internes peuvent d'abord rester regroupés.

Dépôt cible	Package compagnon candidat	Behavior et surface indicative
celestial-core	@spacexr/celestial-core-mcp	CelestialBehavior : catalogue, corps, repères, rotations et éphémérides. AtmosphereModelBehavior selon l'arbitrage du modèle physique.
geodesy_ts	@spacexr/geodesy-mcp	GeodesyBehavior : conversions, projections, ellipsoïdes et repères avec unités explicites.
3d-tiles-core	@spacexr/3d-tiles-core-mcp	TilesetBehavior : inspection et validation des documents et structures implicites.
3d-tiles-core	@spacexr/tiles-mcp	TilePyramidBehavior : adressage, faces, voisins, niveaux et disponibilité des sources.
3d-tiles-core	@spacexr/3d-tiles-runtime-mcp	RuntimeBehavior : état, sélection, budgets, chargements et diagnostics du runtime vivant.
terrain-core	@spacexr/raster-mcp	RasterBehavior : métadonnées, échantillonnage, résolution et références de données.
terrain-core	@spacexr/dem-mcp	DemBehavior : altitude, profil, pente et provenance, selon les fonctions disponibles.
terrain-core	@spacexr/grid-mcp	GridBehavior : topologie, jonctions, niveaux et diagnostics géométriques.
spacexr-rendering	@spacexr/3d-tiles-babylon-mcp	Binding Babylon des behaviors de rendu communs, diagnostics et capacités effectives.
spacexr-rendering	@spacexr/three-tiles3d-mcp	Binding Three des mêmes contrats de rendu ; nom aligné sur le package métier final.

Les autres composants de rendu suivent la même convention, par exemple @spacexr/babylon-terrain-mcp ou @spacexr/babylon-atmosphere-mcp si les packages métier correspondants sont effectivement créés. Les contrats, algorithmes et kernels internes sont décrits par les ressources de capacité et diagnostic ; un compagnon autonome n'est publié que s'il apporte un usage propre.

Les compagnons restent dans le dépôt du composant concerné, en workspace. Aucun dépôt MCP par package n'est nécessaire. Quantia reste hors des cinq dépôts structurants ; un éventuel quantia-mcp demeure optionnel et hors du premier lot.

Grammaires et contrats MCP

Référence technique locale : @cyanmycelium/mcp-core 1.4.1. La grammaire est une couche de descriptions et de formulations ; les schémas JSON définissent les arguments et résultats. Elle ne constitue ni un parseur de langage naturel, ni un moteur d'exécution, ni une politique d'autorisation.

Structure des behaviors

Les behaviors étendent McpBehavior et définissent _buildTools(), _buildResources(), _buildTemplate() et _buildGrammars() selon leur surface. Les bindings étendent McpAdapterBase ou implémentent IMcpBehaviorAdapter. L'exécution passe par executeToolAsync et la lecture par readResourceAsync. Les entrées Node restent dans l'hôte ou un point d'entrée /node ; aucun import /node dans les bundles navigateur.

Chaque behavior embarque les profils default:en et default:fr. Les variantes par client restent facultatives. Les fichiers utilisent la forme structurée McpGrammarData : server, tools, resources, templates et phrases. Les noms d'outils et les identifiants restent stables ; title, description et descriptions de propriétés sont localisables.

Résolution et validation

Ordre de priorité croissante : behavior, adaptateur, configuration statique de l'hôte, store de grammaires. La résolution de locale et de client utilise la politique de mcp-core ; les règles de repli sont testées. Une modification de grammaire ne change ni les types, ni les unités, ni le sens d'un outil.

Les grammaires sont validées avec McpGrammar.check. La règle withWordingRule("default:en") impose un seul emplacement pour chaque description : schéma inline ou grammaire. Le choix retenu ici est la grammaire pour les descriptions. Les phrases traduites conservent leurs clés et paramètres de substitution. Le chargement depuis le disque via loadGrammarDirectory est réservé au point d'entrée Node ; les behaviors navigateur embarquent leurs données.

Identités et résultats

Préfixer les noms pour éviter les collisions : tileset_validate, tiles_address, runtime_getState, dem_sampleElevation et render_getCapabilities sont des exemples de conventions, non un catalogue déjà implémenté. Les URI de composants peuvent suivre spacexr://components/{componentId}/instances/{instanceId}. Les objets d'une scène sont résolus dans la session cible, avec sessionId et instanceId explicites si nécessaire.

Les résultats déclarent un outputSchema lorsque pertinent et renvoient structuredContent avec une représentation textuelle compatible. Les erreurs opérationnelles sont explicites. Un échantillon terrain indique corps, repère, unités, datum vertical, résolution, source et absence éventuelle de données. Les identifiants de tuiles du runtime restent éphémères ; ils ne servent pas d'identité métier durable.

Les ressources exposent configuration, capacités, états et diagnostics bornés. Les rasters, géométries et captures volumineux sont référencés par URI ou artefact. Les handles moteur et GPU restent privés à leurs adaptateurs.

Intégration MCP et critères de sortie

Capacités et cycle de vie

getToolSupport annonce Full ou Partial lorsque l'exécution existe, avec les limites documentées. Planned et None restent masqués. Un profil de grammaire ne peut rendre une opération disponible. Le contrôle d'accès de la session reste distinct de ce mécanisme de capacité.

Les changements passent par _forwardResourceContentChanged(uri) et _forwardResourceChanged() ; mcp-core gère les notifications et abonnements de session. Les états modifiés sont invalidés avant relecture. La télémétrie de rendu est regroupée ou limitée en fréquence, sans notification à chaque frame. Les adaptateurs libèrent leurs abonnements en fin de vie.

Les ressources dépendant du demandeur ne sont pas placées dans le cache partagé de la ressource racine du behavior. Les métadonnées de IMcpRequestContext restent des entrées non fiables ; l'hôte les valide et applique ses autorisations avant toute mutation. Les accès directs et MCP partagent le même mécanisme de modification et de révision de l'état.

Les actions longues utilisent les opérations applicatives existantes : identifiant, progression, état terminal et annulation propagée aux API qui la supportent. accepted ne signifie pas completed. Le mcp-core local déclare tasks/* non pris en charge ; ne pas présenter ces opérations comme des tâches protocolaires MCP ni supposer une propagation AbortSignal fournie par executeToolAsync.

Composition avec Explorer

Explorer possède déjà ExplorerBehavior, ExplorerAdapter, des grammaires FR/EN et des outils par domaine. Conserver ses noms et URI publics ; déléguer progressivement les opérations aux companions. Chaque opération a un propriétaire unique. Le même outil n'est pas enregistré deux fois par Explorer et un composant. Le broker reste une dépendance de l'hôte, via mcp-broker-provider lorsque nécessaire.

Le premier chemin comprend inspection et validation du Tileset, lecture de la pyramide, observation de la sélection réelle du runtime, échantillonnage DEM et capacités de rendu. Il s'exécute sur les mêmes instances que l'application et doit produire les mêmes résultats métier par API directe et par MCP.

Validation avant publication

- Aucun import MCP dans les packages métier ; aucun import Babylon ou Three dans les behaviors génériques. Un utilisateur du package métier n'installe pas MCP par dépendance inverse.
- Chaque composant fonctionnel possède un behavior et une grammaire documentés ; seuls les outils implémentés sont annoncés. Les packages internes sans surface autonome restent regroupés.
- Tests des schémas, descriptions, repli FR/EN, collisions de noms, routage des URI et gestion des instances ; tests serveur-client avec LoopbackTransport.
- Tests de notifications, isolation de session, erreurs et libération des abonnements. Tests de progression et annulation pour les opérations effectivement prises en charge.
- Même sélection du runtime et mêmes résultats DEM avec Babylon et Three à données et caméra normalisée identiques, sans duplication du runtime.
- Vérification des bundles navigateur, des peerDependencies MCP/moteur et du contenu des archives NPM ; publication seulement après validation des versions compatibles.

Sources locales examinées : mcp-core/AGENTS.md et README.md (Grammar layer) ; src/mcp.behavior.ts ; src/mcp.grammar.ts ; src/interfaces/mcp.behavior.interfaces.ts ; spacexr-explorer/src/mcp.ts. Les noms de packages, behaviors et outils de cette extension sont des cibles de migration, sans implémentation ni publication effectuée.